

Lung Segmentation on Chest X-Ray images with U-Net

```
In [1]: from google.colab import drive
drive.mount('/content/drive')
import os
os.chdir('/content/drive/MyDrive/Colab Notebooks/hands_on_ML/files_ML_7_SOL')
```

Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True).

```
In [2]: dim = 128 # Available dimensions: 128, 256, 512

# Allow for overriding in tests
dim = os.environ.get("UNET_SHEZEN_DIM", dim)

data_url = {128: '0e1e5d/d1/Shezen_128128.zip', 256: '3a4a44/d1/Shezen_256256.zip', 512: '80a4eb/d1/Shezen_512512.zip'}

if not os.path.exists(f"./Shezen_{dim:d}_{dim:d}"):
    ! wget https://pandora.infn.it/public/{data_url[dim]} -qO tmp.zip
    ! unzip -q tmp.zip
    ! rm -rf tmp.zip

path='./Shezen_{:d}_{:d}/'.format(dim,dim) # input images path
data_path= path + '**'
```

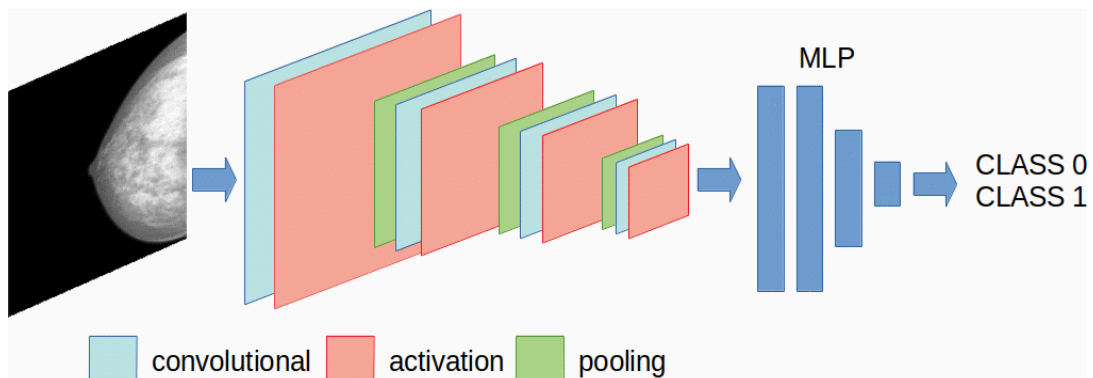
Chest X-Ray (CXR) imaging is a radiographic procedure that produces 2D images of the chest. A typical CXR looks like this:



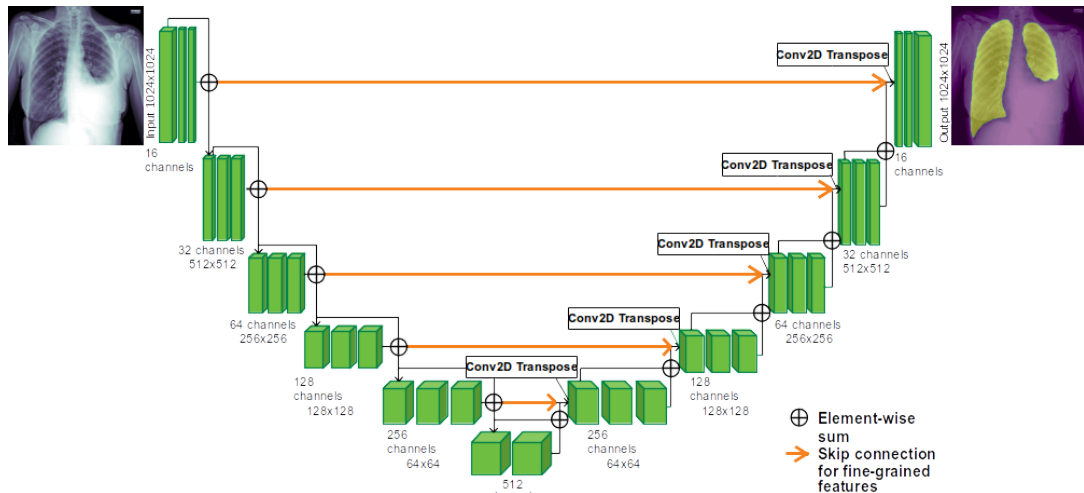
It is clear that there are many pixels that do not belong to lungs, so it is important to delete the useless information that may be contained in the image. In other words, we want to segment the lungs.

In this exercise you will learn how to build, train and test a U-Net for lung segmentation. The network we are going to implement can be easily adapted to any other segmentation problem. As mentioned, a [U-net](#) is a Fully Convolutional Neural Network (FCNN) and it is the state-of-the-art algorithm for medical segmentation. The term FCNN means that there are no dense layers (like in an MLP) at the end of the Network. Try to understand what this means for the network:

- Suppose we have a standard CNN for binary classification of images. We expect the network architecture to be made by several convolutional, pooling or strided convolutional layers, activation layers and so on. At a certain point the input will be flattened by using a simple flattening or a global average pooling, for example. Once the size of the tensor is reduced to one dimension, we use dense layers to let our CNN classify the input images:



- In a FCNN, we perform a pixel-wise classification: image segmentation means to give to each pixel a class, in our case "lung" or "background". The structure of the U-Net mirrors the structure of an autoencoder: the left part, also called **compression path**, is an encoder that compresses the information with a CNN and the right part, called **decompression path**, is a decoder that upsamples the encoded information in order to return an image with the same size as the input one. The name U-Net is due to the fact that the whole architecture can be represented with a U shape



In this exercise, you will learn to build, train and evaluate a CNN for lung segmentation on Chest X-Ray images.

Note: the images have already been pre-processed since their pre-processing requires time (about 2 hours). For your knowledge, the datasets we are going to use were collected to study tuberculosis. These datasets contain Chest X-Ray images and lung masks. The first one is the *Shenzhen dataset*, containing CXR images acquired as part of the routine care at Shenzhen Hospital (Guangdong Medical College, China) and released in JPEG format. There are 340 normal X-Rays and 275 abnormal X-Rays showing various manifestations of tuberculosis. You can download this dataset from the following url:

- 128x128 (<https://pandora.infn.it/public/f10e06>)
- 256x256 (<https://pandora.infn.it/public/88b1cf>)
- 512x512 (<https://pandora.infn.it/public/ced7f7>)

The second one is the *Montgomery dataset*, which has been collected in collaboration with the Department of Health and Human Services, Montgomery County, Maryland, USA. The set contains 138 frontal chest X-Rays from Montgomery County's Tuberculosis screening program, of which 80 are normal cases and 58 are cases with manifestations of TB.

We aggregated the two datasets and pre-processed the images this way:

- images underwent a contrast stretching between their 5th and 95th percentile and normalized.
- some images presented black borders, so we cropped and resized them to 256x256 size.

In this exercise, we do not apply data augmentation because it will increase training time. You can develop your data augmentation in real-time during training as a further advancement.

Also, a peculiarity of the U-Net are the skip connections: the output of a left block is concatenated to the input of the correspondent right block. So, when we write the architecture we should keep an eye on the sizes of the outputs of the compression path and the inputs of the decompression path: they have to be the same! A smart choice to match the sizes is to use the same size for stride/pooling on the left path and for the upsampling/deconvolution in the right path.

Finally, it is important to let the U-Net have a good generalization capability: we will be using both batch normalization and spatial dropout (at the end of the block).

Data Generator

When we deal with a small amount of data we can use different ways to upload it and give it as input to a NN (for example it can be loaded in a `numpy` array). As said, when we deal with some medical images, sometimes it is not possible to use this modality because we will rapidly saturate the RAM. For this reason we can use a *data generator* based on the `Keras` class `Sequence` to load only one batch per iteration instead of loading the entire dataset at once.

In Keras it is possible to use a simple built in generator, called `ImageDataGenerator`, which is a generator that can be used for classification and segmentation task. However, we will be building our own, since the goal of this part of the exercise is to broadly understand its functioning. We basically have to write an iterator that allows us to choose the couple of input data image and label.

Let's import the packages we need:

```
In [3]: from tensorflow.keras.utils import Sequence
import numpy as np
import os
import tensorflow as tf
from glob import glob
import matplotlib.pyplot as plt
```

A `DataGenerator` is a class defined through the class `Sequence`, so let's build it. This code contains comments that are useful to understand each single part of the `DataGenerator`. There are also a few useful questions with corresponding answers summarized below the definition.

```
In [4]: import numpy as np
import os
from keras.utils import Sequence
import tensorflow as tf

class DataGenerator(Sequence):
    def __init__(self, path, list_X, batch_size=1, dim=(128,128), shuffle=True, **kwargs):
        super().__init__(**kwargs) # added just to silence a warning
        self.dim = dim
        self.batch_size = batch_size
        self.list_X = list_X
        self.path = path
        self.shuffle = shuffle
        self.on_epoch_end()

    def __len__(self):
        return len(self.list_X) // self.batch_size

    def __getitem__(self, index):
        indexes = self.indexes[index * self.batch_size:(index + 1) * self.batch_size]
        list_X_temp = [self.list_X[k] for k in indexes]
        return self._data_generation(list_X_temp)

    def on_epoch_end(self):
        self.indexes = np.arange(len(self.list_X))
        if self.shuffle:
            np.random.shuffle(self.indexes)

    def _data_generation(self, list_X_temp):
        X = np.empty((self.batch_size, *self.dim, 1), dtype=np.float32)
        y = np.empty((self.batch_size, *self.dim, 1), dtype=np.float32)

        for i, j in enumerate(list_X_temp):
            arr = np.load(os.path.join(self.path, str(j)))
            X[i] = np.expand_dims(arr[:, :, 0], axis=-1)
            y[i] = np.expand_dims(arr[:, :, 1], axis=-1)

        return X, y
```

QUESTIONS:

1) When are the image indices reshuffled?

The image indices are reshuffled at two specific moments:

- At the very beginning, when the `DataGenerator` object is created (because `self.on_epoch_end()` is explicitly called inside the `__init__` method).
- At the end of every training epoch (Keras automatically calls the `on_epoch_end()` method when an epoch finishes).

2) Why indices reshuffling is so important?

Reshuffling prevents the neural network from memorizing the order of the data. If the model sees the data in the exact same sequence every epoch, it might learn patterns based on the order rather than the actual features of the images.

Shuffling ensures that each batch has a random and diverse set of samples, which leads to smoother gradient updates and helps the model generalize better to unseen data, reducing overfitting.

3) Where do we set `shuffle=True` ?

It is set as a default parameter in the constructor of the class: `def __init__(..., shuffle=True)`. When you instantiate the generator for the training set (e.g., `train_gen = DataGenerator(...)`), it will automatically be `True`. You would explicitly set it to `False` when creating the generator for the validation or test sets, where order matters for evaluation.

4) What is the dimension of `List_X_temp` ?

The length of `list_X_temp` is equal to `self.batch_size` (which defaults to 1 in this code). It is a simple list containing the filenames (or paths) of the samples that will be loaded for the current batch.

5) We could perform also data augmentation with the above generator. Where should it be performed?

Data augmentation should be performed inside the `_data_generation` method, right inside the `for` loop. After loading the image array (`arr = np.load(...)`), you would apply the augmentation transformations to the array before assigning the values to `X[i]` and `y[i]`. *Important note:* For tasks like image segmentation, spatial transformations (like rotations or flips) must be applied identically and simultaneously to both the image (`X`) and its mask (`y`).

6) What happens if we implement data augmentation in the generator?

If implemented in the generator, data augmentation happens "on-the-fly" dynamically during training. This has two huge advantages:

- *Memory Efficiency:* You don't need to generate and save thousands of augmented images on your hard drive, saving massive amounts of storage.
- *Infinite Variations:* The model sees a slightly different, newly augmented version of the original image at every single epoch, making the network much more robust and heavily reducing overfitting.

Now that we have written the data generator, let's test if it works properly.

```
In [5]: data_path = os.path.join(path, '*') # List of images name
data_list = [os.path.basename(f) for f in glob(data_path)]

# Selecting the first 10 patients for the test
data_list = data_list[0:10]

print(data_list)

['CHNCXR_0272_0_mask.npy', 'CHNCXR_0380_1_mask.npy', 'CHNCXR_0061_0_mask.npy', 'CHNCXR_0317_0_mask.npy', 'CHNCXR_0396_1_mask.npy', 'CHNCXR_0077_0_mask.npy', 'MCUCXR_0094_0.npy', 'CHNCXR_0081_0_mask.npy', 'CHNCXR_0434_1_mask.npy', 'CHNCXR_0276_0_mask.npy']
```

```
In [6]: dg_test = DataGenerator(path, list_X=data_list, batch_size = 1) # we call the data generator
```

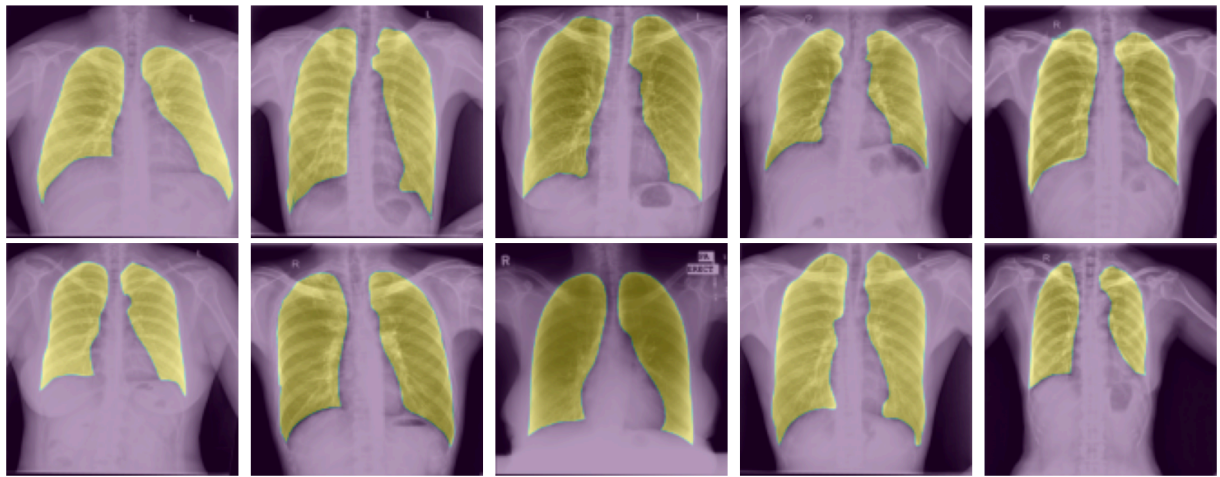
We plot the images with the labels of the selected cases.

```
In [7]: fig, axes = plt.subplots(2, 5, figsize=(15, 6))
axes = axes.flatten()

for i in range(min(10, dg_test.__len__())):
    X, Y = dg_test.__getitem__(index=i)

    axes[i].imshow(X[0,:,:,:], cmap='gray')
    axes[i].imshow(Y[0,:,:,:], alpha=0.4)
    axes[i].axis('off')
    #axes[i].set_title(data_list[i], fontsize=9)

plt.tight_layout()
```



U-Net architecture definition

Keeping in mind what we said in the introductory part of the notebook, we will write three functions, one for the decompression path, one for the bottleneck block and one for the compression path. Then we will put them together to write the whole U-Net architecture.

Let's start by importing some other useful packages.

```
In [8]: import numpy as np
import os
from tensorflow.keras import backend as K
from tensorflow.keras import regularizers, initializers
from tensorflow.keras.activations import softmax
from tensorflow.keras.layers import Input, BatchNormalization, SpatialDropout2D, Conv2D, Conv2DTranspose, Concatenate
from tensorflow.keras.models import Sequential, Model
import tensorflow as tf

os.environ['TF_CPP_MIN_LOG_LEVEL'] = "3"
os.environ["CUDA_VISIBLE_DEVICES"] = "0"
```

A) Compression path

Take a look at the code below where the compression path is defined. Here are some things that you should notice about it:

- *Regularizer*: We can define a penalty on layer parameters or layer activity during optimization. In the following, we use `kernel_regularizer`, which means we are applying a penalty on the kernel layers.
- *Initializer*: When we begin the training of a NN we can set the initial weights distribution. In this exercise, both regularizers and initializers (`'random_normal'`) are declared inside the convolutional layer.
- *Input*: Input is a function that is used to instantiate a Keras tensor. A Keras tensor is a symbolic tensor-like object, which we augment with certain attributes that allow us to build a Keras model just by knowing the inputs and outputs of the model. For example attributes are: input shape, batch size... You can find the full attributes list here: https://keras.io/api/layers/core_layers/input/.
- *BatchNormalization*: Batch normalization (see <https://arxiv.org/abs/1502.03167>) is a way to regularize the model by reducing the internal covariate shift. This layer normalizes its output using the mean and standard deviation of the current batch of inputs and maintains the mean close to zero and the standard deviation close to 1.
- *SpatialDropout2D*: As said, dropout is a method used to regularize the NN. It randomly sets input units to 0 with a frequency of rate at each step during training time, which helps prevent overfitting. `SpatialDropout2D` (see <https://arxiv.org/abs/1411.4280>) is instead applied on an entire feature map. This helps the network to learn independent feature maps.
- *Conv2D*: This defines a 2D Convolution Layer. At this url you can find the parameters we can set in the convolutional layers: https://keras.io/api/layers/convolution_layers/convolution2d/.

```
In [9]: def unet_layer_left(
previous_layer,
n_filters,
```

```

ker_size_1,
strides_1,
ker_size_2,
strides_2,
ker_size_3,
strides_3,
reg,
):
    """Definition of layers of the left part of the u-net.

    Parameters
    -----
    previous_layer : input layer for this block;
    n_filters : number of filters of all the 2D convolutional layers;
    ker_size_1 : kernel dimension of the first Conv2D layer of this block;
    strides_1 : strides size to be used for the convolution of the first Conv2D layer;
    ker_size_2/3 : kernel dimension of the second/third Conv2D layers;
    strides_2/3 : strides size to be used for the convolution of the second/third Conv2D layer;
    """
    layer_L = Conv2D(
        filters=n_filters,
        kernel_size=ker_size_1,
        strides=strides_1,
        kernel_regularizer=regularizers.l2(reg),
        kernel_initializer='random_normal'
    )(previous_layer)

    layer_L = BatchNormalization(axis=-1)(layer_L)
    layer_L_shortcut = layer_L
    layer_L = Activation('relu')(layer_L)
    layer_L = SpatialDropout2D(0.2)(layer_L)

    layer_L = Conv2D(
        filters=n_filters,
        kernel_size=ker_size_2,
        strides=strides_2,
        padding='same',
        kernel_regularizer=regularizers.l2(reg),
        kernel_initializer='random_normal'
    )(layer_L)

    layer_L = BatchNormalization(axis=-1)(layer_L)
    layer_L = Activation('relu')(layer_L)
    layer_L = SpatialDropout2D(0.2)(layer_L)

    layer_L = Conv2D(
        filters=n_filters,
        kernel_size=ker_size_3,
        strides=strides_3,
        padding='same',
        kernel_regularizer=regularizers.l2(reg),
        kernel_initializer='random_normal'
    )(layer_L)

    layer_L = BatchNormalization(axis=-1)(layer_L)
    layer_L = Add()([layer_L, layer_L_shortcut])
    layer_L = Activation('relu')(layer_L)
    layer_L = SpatialDropout2D(0.2)(layer_L)

    return layer_L

```

QUESTIONS

1) How many convolution layers are there?

There are exactly 3 convolutional layers in this block, created by calling the `Conv2D` function three separate times.

2) Why we stored the variable 'layer_L' in 'layer_L_shortcut'?

It is used to create a skip connection. By saving the state of the tensor in `layer_L_shortcut` right after the first convolution and batch normalization, the network can bypass the next two convolutional layers. Later, this saved tensor is added back to the main flow using the `Add()([layer_L, layer_L_shortcut])` layer.

3) What is the padding? What does 'same' mean?

Padding is the process of adding an extra border of pixels (usually filled with zeros) around the outside edges of an image or a feature map before applying a convolution operation. Setting ``padding='same'`` tells Keras to automatically calculate and apply just enough padding so that the output feature map has the **same** spatial dimensions (height and width) as the input feature map (assuming `strides=1`). Without padding, every convolution operation would gradually shrink the spatial dimensions of the image.

B) Bottleneck block

The following cell contains the code for the bottleneck block of the U-Net. Now you should understand how it works.

```
In [10]: def unet_layer_bottleneck(
    previous_layer,
    n_filters,
    ker_size_1,
    strides_1,
    ker_size_2,
    strides_2,
    reg,
):
    """Definition of the last layer of the network.

    Parameters
    -----
    previous_layer : input layer for this block;
    n_filters : number of filters of all the 2D convolutional layers;
    ker_size_1 : kernel dimension of the first conv2D layer of this block;
    strides_1 : strides size to be used for the convolution of the first Conv2D layer;
    ker_size_2 : kernel dimension of the second Conv2D layers;
    strides_2 : strides size to be used for the convolution of the second Conv2D layer;
    """
    layer_L = Conv2D(
        filters=n_filters,
        kernel_size=ker_size_1,
        strides=strides_1,
        kernel_regularizer=regularizers.l2(reg),
        kernel_initializer='random_normal'
    )(previous_layer)

    layer_L = BatchNormalization(axis=-1)(layer_L)
    layer_L_shortcut = layer_L
    layer_L = Activation('relu')(layer_L)

    layer_L = Conv2D(
        filters=n_filters,
        kernel_size=ker_size_2,
        strides=strides_2,
        padding='same',
        kernel_regularizer=regularizers.l2(reg),
        kernel_initializer='random_normal'
    )(layer_L)

    layer_L = BatchNormalization(axis=-1)(layer_L)
    layer_L = Add()([layer_L, layer_L_shortcut])
    layer_L = Activation('relu')(layer_L)

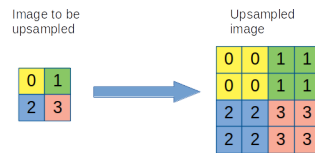
    return layer_L
```

C) Decompression path

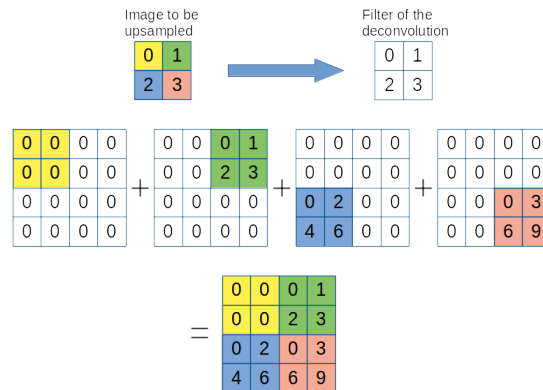
In the following cell the decompression path is defined. As you can see each block of the decompression path takes as input the corresponding left layer in order to build the skip connections. Here are some other noticeable aspects of the code:

- `Conv2DTranspose` : As said many times before, in the first (left) part of the U-Net (strided) convolution, pooling and activations are used to compress the information, while in the second (right) part the compressed information is upsampled to the input size. We can mainly perform upsampling in two ways:

1. Using a simple parameters-free upsampling;



2. Using transpose convolution, with the advantage that the network will have kernels that can learn the better way to upsample the information.



- **Concatenate** : This simply concatenates two tensors. In this case, we are using the concatenate layer to create the skip connections between the left and the right part of the U-Net.
- **Activation** : The activation layer is the one used to define a non-linear function in order to activate the kernel. In this case we are going to use a simple ReLU.
- **Add** : The add layer simply sums two input tensors. In this architecture it is used to compute a residual connection inside each block of both compression and decompression path.

```
In [11]: def unet_layer_right(
    previous_layer,
    layer_left,
    n_filters,
    ker_size_1,
    strides_1,
    output_pad,
    ker_size_2,
    strides_2,
    ker_size_3,
    strides_3,
    reg,
):
    """Definition of layers of the right part of the u-net.

    Parameters
    -----
    previous_layer : input layer for this block;
    layer_left : output layer of the left part at the same depth;
    n_filters : int, number of filters of all the 2D convolutional layers;
    ker_size_1 : int, kernel dimension of the first conv2D layer of this block;
    strides_1 : int, strides size to be used for the convolution of the first Conv2D layer;
    output_pad : tuple, padding for the output;
    ker_size_2/3 : int, kernel dimension of the second/third Conv2D layers;
    strides_2/3 : int, strides size to be used for the convolution of the second/third Conv2D layer;
    """
    layer_R = Conv2DTranspose(
        filters=n_filters,
        kernel_size=ker_size_1,
        strides=strides_1,
        output_padding=output_pad,
        kernel_regularizer=regularizers.l2(reg),
        kernel_initializer='random_normal'
    )(previous_layer)

    layer_R = BatchNormalization(axis=-1)(layer_R)
    layer_R_shortcut = layer_R
    layer_R = Activation('relu')(layer_R)

    merge = Concatenate(axis=-1)([layer_left, layer_R])
```

```

layer_R = Conv2D(
    filters=n_filters,
    kernel_size=ker_size_2,
    strides=strides_2,
    padding='same',
    kernel_regularizer=regularizers.l2(reg),
    kernel_initializer='random_normal'
)(merge)

layer_R = BatchNormalization(axis=-1)(layer_R)
layer_R = Activation('relu')(layer_R)
layer_R = SpatialDropout2D(0.2)(layer_R)

layer_R = Conv2D(
    filters=n_filters,
    kernel_size=ker_size_3,
    strides=strides_3,
    padding='same',
    kernel_regularizer=regularizers.l2(reg),
    kernel_initializer='random_normal'
)(layer_R)

layer_R = BatchNormalization(axis=-1)(layer_R)
layer_R = Add()([layer_R, layer_R_shortcut])
layer_R = Activation('relu')(layer_R)
layer_R = SpatialDropout2D(0.2)(layer_R)

return layer_R

```

QUESTIONS

1) Why is axis set to -1 in the concatenate layer? What does it mean?

In Python and NumPy/TensorFlow, an axis of -1 always refers to the *last dimension* of a tensor. For 2D image data in Keras, the shape of the tensor is (batch_size, height, width, channels). Therefore, axis=-1 means we are concatenating the tensors along the channels axis. In a U-Net, this is exactly what we want to do: we take the feature maps from the contracting path (layer_left) and stack them together with the upsampled feature maps of the expanding path (layer_R), keeping the height and width identical but increasing the depth (the number of filters/channels).

2) What conditions have to be satisfied in order to perform the concatenation?

To concatenate two or more tensors along a specific axis, *all other axes must have the exact same dimensions*. Since we are concatenating along the channels axis (axis=-1), layer_left and layer_R must have:

- The exact same spatial dimensions (same height and same width).
- The same batch_size (which is naturally true during training/testing).

The only dimension that is allowed to be different between the two tensors is the one specified by the axis (the number of channels).

D) Complete U-Net architecture

Now that we have our blocks, let's put everything together.

```

In [12]: def U_net(input_size):
        """This function builds the network without compiling it.

        Parameters
        -----
        input_size : tuple, size of the input
        reg : float, regularization parameters (L2)
        """
        inputs = Input(shape=(input_size)) ##

        Level_1_L = unet_layer_left(
            inputs, n_filters=16, reg=0.1,
            ker_size_1=1, strides_1=1, ker_size_2=3, strides_2=1,
            ker_size_3=1, strides_3=1
        )
        Level_2_L = unet_layer_left(
            Level_1_L, n_filters=32, reg=0.1,
            ker_size_1=2, strides_1=2, ker_size_2=3, strides_2=1,
            ker_size_3=1, strides_3=1
        )

```

```

Level_3_L = unet_layer_left(
    Level_2_L, n_filters=64, reg=0.1,
    ker_size_1=2, strides_1=2, ker_size_2=3, strides_2=1,
    ker_size_3=1, strides_3=1
)
Level_4_L = unet_layer_left(
    Level_3_L, n_filters=128, reg=0.1,
    ker_size_1=2, strides_1=2, ker_size_2=3, strides_2=1,
    ker_size_3=1, strides_3=1
)
Level_5_L = unet_layer_left(
    Level_4_L, n_filters=256, reg=0.1,
    ker_size_1=2, strides_1=2, ker_size_2=3, strides_2=1,
    ker_size_3=1, strides_3=1
)

Level_6_L = unet_layer_bottleneck(
    Level_5_L, n_filters=512, reg=0.1,
    ker_size_1=1, strides_1=1, ker_size_2=3, strides_2=1
)

Level_5_R = unet_layer_right(
    Level_6_L, Level_5_L, n_filters=256, reg=0.1, output_pad=None,
    ker_size_1=1, strides_1=1, ker_size_2=3, strides_2=1,
    ker_size_3=3, strides_3=1
)
Level_4_R = unet_layer_right(
    Level_5_R, Level_4_L, n_filters=128, reg=0.1, output_pad=None,
    ker_size_1=2, strides_1=2, ker_size_2=3, strides_2=1,
    ker_size_3=1, strides_3=1
)
Level_3_R = unet_layer_right(
    Level_4_R, Level_3_L, n_filters=64, reg=0.1, output_pad=None,
    ker_size_1=2, strides_1=2, ker_size_2=3, strides_2=1,
    ker_size_3=1, strides_3=1
)
Level_2_R = unet_layer_right(
    Level_3_R, Level_2_L, n_filters=32, reg=0.1, output_pad=None,
    ker_size_1=2, strides_1=2, ker_size_2=3, strides_2=1,
    ker_size_3=1, strides_3=1
)
Level_1_R = unet_layer_right(
    Level_2_R, Level_1_L, n_filters=16, reg=0.1, output_pad=None,
    ker_size_1=2, strides_1=2, ker_size_2=3, strides_2=1,
    ker_size_3=1, strides_3=1
)

output = Conv2D(
    filters=1, kernel_size=1, strides=1,
    activation='sigmoid', kernel_regularizer=regularizers.l2(0.1)
)(Level_1_R)

model = Model(inputs=inputs, outputs=output)

return model

```

Let's now test the architecture by generating a set of mock data and training the U-Net on it.

```

In [13]: # Mock data
test_data = np.zeros((2,256,256,1))
test_label = np.zeros((2,256,256,1))
# Calling the U-Net function
model = U_net((256,256,1))

In [14]: # Choose a built-in metric in keras that is Intersection Over Union
met = tf.keras.metrics.MeanIoU(num_classes=2)
# Define an optimizer and a Loss function
model.compile(optimizer="rmsprop", loss="sparse_categorical_crossentropy")
#model.summary() # Uncomment this line to print the model
# Fit the model
history=model.fit(test_data, test_label, epochs=3, verbose=0)

```

What if the input size is not a power of 2? Let's try it:

```

In [15]: test_data = np.zeros((2,200,200,1))
test_label = np.zeros((2,200,200,1))

```

```
In [16]: model = U_net((200,200,1))
```

```
-----
ValueError                                Traceback (most recent call last)
/tmp/ipykernel_30067/1145076899.py in <cell line: 0>()
----> 1 model = U_net((200,200,1))

/tmp/ipykernel_30067/2886919062.py in U_net(input_size)
    45     ker_size_3=3, strides_3=1
    46 )
--> 47 Level_4_R = unet_layer_right(
    48     Level_5_R, Level_4_L, n_filters=128, reg=0.1, output_pad=None,
    49     ker_size_1=2, strides_1=2, ker_size_2=3, strides_2=1,

/tmp/ipykernel_30067/3864781529.py in unet_layer_right(previous_layer, layer_left, n_filters, ker_size_1, strides_1, output_pad, ker_size_2, strides_2, ker_size_3, strides_3, reg)
    38     layer_R = Activation('relu')(layer_R)
    39
--> 40     merge = Concatenate(axis=-1)([layer_left, layer_R])
    41
    42     layer_R = Conv2D(

/usr/local/lib/python3.13/dist-packages/keras/src/utils/traceback_utils.py in error_handler(*args, **kwargs)
    120         # To get the full stack trace, call:
    121         # `keras.config.disable_traceback_filtering()`
--> 122         raise e.with_traceback(filtered_tb) from None
    123     finally:
    124         del filtered_tb

/usr/local/lib/python3.13/dist-packages/keras/src/layers/merging/concatenate.py in build(self, input_shape)
    97         )
    98         if len(unique_dims) > 1:
--> 99             raise ValueError(err_msg)
   100
   101     def _merge_function(self, inputs):

ValueError: A `Concatenate` layer requires inputs with matching shapes except for the concatenation axis. Received: input_shape=[(None, 25, 25, 128), (None, 24, 24, 128)]
```

QUESTIONS

1) What does this error mean?

The `ValueError` means that the network is trying to concatenate two tensors that have different spatial dimensions. As said, the `Concatenate` layer requires everything to match exactly, except for the axis you are concatenating along. In this specific case, it received one tensor that is 25x25 and another that is 24x24, so it crashed.

2) Why the input shapes of the `Layer_4_L` have different values?

This happens because the initial size (200x200) is being repeatedly halved by the `strides=2` in the convolutional layers. Let's trace the spatial dimensions step-by-step:

- Level 1: 200x200
- Level 2: 100x100
- Level 3: 50x50
- Level 4 (Left): **25x25**
- Level 5 (Left): 25 / 2 = 12.5.

Since pixels can't be fractions, the convolution operation drops the remainder, outputting a **12x12** tensor. When the network moves to the right side (the expansive path) and starts upsampling, it multiplies the dimensions by 2. When it upsamples the 12x12 tensor from Level 5 back to Level 4, it does: 12 * 2 = **24x24**. Finally, it tries to concatenate the upsampled 24x24 tensor with the original skip-connection tensor from Level 4_L which was 25x25. Because 24 != 25, the code breaks.

3) If the input image size is not a power of 2, what parameters we should check in the convolutional layers in order to not have errors?

If your input size is not a power of 2 (or more accurately, not a multiple of 2^N , where N is the number of downsampling steps), you will face this mismatch. To fix it, you need to check and use the `output_pad` (which maps to the `output_padding` parameter in the `Conv2DTranspose` layer). By setting the correct `output_padding`, you can force the upsampling layer to add that "missing" pixel back (e.g., forcing it to output 25x25 instead of 24x24). Alternatively, to

avoid checking parameters entirely, you can simply ensure your input image size is padded or resized beforehand so that it is evenly divisible by 2^N (in this network, there are 4 downsampling steps with stride 2, so the input size must be divisible by $2^4 = 16$. Notice that 256 is divisible by 16, but 200 is not).

```
In [17]: def U_net_adj(input_size):
    """This function builds the network without compiling it.

    Parameters
    -----
    input_size : tuple, size of the input
    reg : float, regularization parameters (L2)
    """
    inputs = Input(shape=(input_size)) ##

    Level_1_L = unet_layer_left(
        inputs, n_filters=16, reg=0.1,
        ker_size_1=1, strides_1=1, ker_size_2=3, strides_2=1,
        ker_size_3=1, strides_3=1
    )
    Level_2_L = unet_layer_left(
        Level_1_L, n_filters=32, reg=0.1,
        ker_size_1=2, strides_1=2, ker_size_2=3, strides_2=1,
        ker_size_3=1, strides_3=1
    )
    Level_3_L = unet_layer_left(
        Level_2_L, n_filters=64, reg=0.1,
        ker_size_1=2, strides_1=2, ker_size_2=3, strides_2=1,
        ker_size_3=1, strides_3=1
    )
    Level_4_L = unet_layer_left(
        Level_3_L, n_filters=128, reg=0.1,
        ker_size_1=2, strides_1=2, ker_size_2=3, strides_2=1,
        ker_size_3=1, strides_3=1
    )
    Level_5_L = unet_layer_left(
        Level_4_L, n_filters=256, reg=0.1,
        ker_size_1=2, strides_1=2, ker_size_2=3, strides_2=1,
        ker_size_3=1, strides_3=1
    )

    Level_6_L = unet_layer_bottleneck(
        Level_5_L, n_filters=512, reg=0.1,
        ker_size_1=1, strides_1=1, ker_size_2=3, strides_2=1
    )

    Level_5_R = unet_layer_right(
        Level_6_L, Level_5_L, n_filters=256, reg=0.1, output_pad=None,
        ker_size_1=1, strides_1=1, ker_size_2=3, strides_2=1,
        ker_size_3=3, strides_3=1
    )
    Level_4_R = unet_layer_right(
        Level_5_R, Level_4_L, n_filters=128, reg=0.1, output_pad=(1, 1),
        ker_size_1=2, strides_1=2, ker_size_2=3, strides_2=1,
        ker_size_3=1, strides_3=1
    )
    Level_3_R = unet_layer_right(
        Level_4_R, Level_3_L, n_filters=64, reg=0.1, output_pad=None,
        ker_size_1=2, strides_1=2, ker_size_2=3, strides_2=1,
        ker_size_3=1, strides_3=1
    )
    Level_2_R = unet_layer_right(
        Level_3_R, Level_2_L, n_filters=32, reg=0.1, output_pad=None,
        ker_size_1=2, strides_1=2, ker_size_2=3, strides_2=1,
        ker_size_3=1, strides_3=1
    )
    Level_1_R = unet_layer_right(
        Level_2_R, Level_1_L, n_filters=16, reg=0.1, output_pad=None,
        ker_size_1=2, strides_1=2, ker_size_2=3, strides_2=1,
        ker_size_3=1, strides_3=1
    )

    output = Conv2D(
        filters=1, kernel_size=1, strides=1,
        activation='sigmoid', kernel_regularizer=regularizers.l2(0.1)
    )(Level_1_R)

    model = Model(inputs=inputs, outputs=output)
```

```
return model
```

```
In [18]: model = U_net_adj((200,200,1))
```

```
In [19]: met = tf.keras.metrics.MeanIoU(num_classes=2)
model.compile(optimizer="rmsprop", loss="sparse_categorical_crossentropy")
#model.summary()
history=model.fit(test_data, test_label, epochs=3, verbose=0)
```

Loss functions and metrics

As any supervised training we have to define the loss functions and the metrics we want to use to train the algorithm.

The problem we want to solve is a segmentation problem. For this reason we should use a correct loss function and metric to both optimize and evaluate the training.

Regarding the loss function, we are going to use the Dice Similarity Coefficient (DSC) that measures how much the true and the predicted areas overlap. DSC is defined this way:

$$DSC(M_{true}, M_{pred}) = 2 \cdot \frac{M_{true} \cdot M_{pred}}{M_{true} + M_{pred}}$$

Visually:

$$DSC(M_{true}, M_{pred}) = 2 \times \frac{\text{Intersection}}{\text{Union}}$$

If there is a perfect overlapping, the DSC will be equal to 1 and there is none, DSC will be equal to 0. In this case we want to use the DSC as a loss function, so the better the overlapping the nearer it must be to 0. So we define the DSC loss function like this:

$$DSC_{loss}(M_{true}, M_{loss}) = 1 - 2 \cdot \frac{M_{true} \cdot M_{pred}}{M_{true} + M_{pred}}$$

Let's implement it using `tensorflow`.

```
In [20]: import tensorflow as tf

# Define dice_loss and dice_coef
def dice_loss(y_true, y_pred, smooth=1e-6):
    y_true_f = tf.cast(y_true, tf.float32)
    y_pred_f = tf.cast(y_pred, tf.float32)

    # Assuming input shape is (batch, height, width, channels),
    # the axes to sum over for spatial and channel dimensions are 1, 2, 3.
    axes = (1, 2, 3)

    intersection = tf.reduce_sum(y_true_f * y_pred_f, axis=axes)
    cardinal_true = tf.reduce_sum(y_true_f, axis=axes)
    cardinal_pred = tf.reduce_sum(y_pred_f, axis=axes)

    dice_coefficient = (2. * intersection + smooth) / (cardinal_true + cardinal_pred + smooth)
    return 1. - dice_coefficient

def dice_coef(y_true, y_pred, smooth=1e-6):
    y_true_f = tf.cast(y_true, tf.float32)
    y_pred_f = tf.math.round(tf.cast(y_pred, tf.float32))

    # Same fix as in dice_loss_fixed
    axes = (1, 2, 3)

    intersection = tf.reduce_sum(y_true_f * y_pred_f, axis=axes)
    cardinal_true = tf.reduce_sum(y_true_f, axis=axes)
    cardinal_pred = tf.reduce_sum(y_pred_f, axis=axes)
```

```

dice_coefficient = (2. * intersection + smooth) / (cardinal_true + cardinal_pred + smooth)
return dice_coefficient

```

We can use many different loss function to solve this problem but for now DSC loss will do. In the same way, we can define many metrics to measure the performance of our algorithm. For now, we will see only the DSC explained above or the Jaccard coefficient. The **Jaccard index** or **Jaccard similarity coefficient** is an index that measures the similarity between two samples and it is simply defined as the intersection over union:

$$Jaccard(M_{true}, M_{pred}) = \frac{M_{true} \cdot M_{pred}}{M_{true} + M_{pred} - M_{true} \cdot M_{pred}}$$

*QUESTIONS

1) We want to obtain a binary mask but we cannot perform the rounding operation in the loss function. Why?

We cannot use a rounding operation (like `tf.math.round`) inside the loss function because *rounding is non-differentiable* (its derivative is exactly zero almost everywhere). Neural networks learn through backpropagation, which requires calculating the gradients (derivatives) of the loss function to update the model's weights. If you round the predictions to exactly 0 or 1, the function becomes a rigid step. The gradient of a flat step is zero and backpropagation breaks down. The optimizer receives no feedback on how to adjust the weights, and the network completely stops learning.

This is why `dice_loss` uses the continuous, raw probabilities (floating-point numbers between 0 and 1, output by the sigmoid activation) to maintain a smooth, continuous gradient flow.

Conversely, `dice_coef` is an evaluation metric used to check the model's real-world accuracy. Since evaluation metrics are only for our observation and do not participate in backpropagation, we can safely apply rounding there to see how the final binary masks will actually perform.

Training the UNET

We have defined the data generator, the architecture, the loss function and the metric we are going to use. Now, we put all the pieces together to train our U-Net. First, we import the libraries we need for the training.

```

In [21]: import numpy as np
import tensorflow as tf
import json
import os
os.environ['TF_CPP_MIN_LOG_LEVEL'] = '3'
os.environ['CUDA_VISIBLE_DEVICES']="0"
from glob import glob

from tensorflow.keras import initializers, regularizers, constraints, optimizers, metrics
from tensorflow.keras import backend as K
from tensorflow.keras.activations import softmax
from tensorflow.keras.layers import Dense, Input, Conv2D, Conv2DTranspose, Dropout, Flatten, BatchNormalizati
from tensorflow.keras.models import Sequential, Model
from tensorflow.keras.callbacks import ModelCheckpoint, ReduceLROnPlateau
from tensorflow.keras.utils import Sequence

#import Data_Generator.DataGenerator3 as DG
#import Loss_Metrics.LossMetrics3
#import UNet_Arch.UNET_architecture as unet

tf.keras.backend.clear_session()

```

We set the path and the names of the data set. We downloaded the data and now we want to split them into training, validation and test set. If you run again this notebook, you might want to leave the data partition as written such that each group will test their architecture on the same test set.

In the following cell we are setting the data generator for both the training and the validation set.

```

In [22]: data_list = [os.path.basename(f) for f in glob(data_path)]

train_list = data_list[0:500]
val_list = data_list[500:590]
test_list = data_list[590:]
training_gen = DataGenerator(path=path,list_X=train_list,batch_size=8,dim=(dim,dim))
validation_gen = DataGenerator(path=path,list_X=val_list,batch_size=1,dim=(dim,dim))

```

As you may know, we can use some built-in callbacks that help to train the network. `ModelCheckpoint` is used to save the weights at the epoch with the best validation metric. You need to create a folder named 'results'. You can read the documentation https://keras.io/api/callbacks/model_checkpoint/.

`ReduceLROnPlateau` is used to decrease the learning rate during training. You can read the documentation https://keras.io/api/callbacks/reduce_lr_on_plateau/.

Of course, we also define the optimizer.

```
In [23]: os.makedirs('results', exist_ok=True)

#checkpoint_path="results/weights-{epoch:03d}-{loss:.2f}-{DSC:.2f}.hdf5" # intermediate weight save path
checkpoint_path_val="results/weights-val-{epoch:03d}-{loss:.2f}-{val_dice_coef:.2f}.weights.h5" # intermedia

#check=ModelCheckpoint(filepath=checkpoint_path, monitor='loss', verbose=0, save_best_only=False, save_weight
check_val=ModelCheckpoint(filepath=checkpoint_path_val, monitor='val_dice_coef', verbose=0, save_best_only=Tr

reduce_lr = ReduceLROnPlateau(monitor='val_dice_coef', factor=0.5, patience=8, min_lr=0.000005, verbose= True

adamlr = optimizers.Adam(learning_rate=0.0001, beta_1=0.9, beta_2=0.999, epsilon=1e-08, amsgrad=True)
```

Now we call the U_Net architecture, compile and train it.

```
In [24]: unet=U_net((dim,dim,1))
unet.compile(loss=dice_loss,optimizer=adamlr, metrics=[dice_coef])
#unet.summary()
history = unet.fit(training_gen, validation_data = validation_gen, epochs=50, verbose=0, callbacks=[check_val

with open('history.json', 'w') as f:
    json.dump(str(history.history), f)
```

Prediction computation and evaluation

Once the neural network has been trained, one can use the fitted weights to infer a prediction of an image that the model has never seen. In this stage we can test the predictive power of the network by comparing the inferred results with previously labelled data. Here we will define a variable containing the model where we can load the trained weights and subsequently use our test data to evaluate our model. Once this is done we store the results in an array so we can plot which image has the best prediction and compare it with the worst prediction. To define which is the most/least accurate prediction we'll have to use some metrics and, as said, here we'll use the dice metric. Furthermore, before doing any inference we will check how the training went, so that we may have an idea on the final performance on the test set.

First, let's start by *evaluating the training*. To do so, we will read the `history.json` file written before and extract the information to be plotted.

```
In [38]: history_dict = json.load(open("history.json", 'r'))
#print(history_dict)

In [26]: history_dict = json.load(open("history.json", 'r'))
history_dict = json.loads(r"{}".format(history_dict).replace("\'", "'").replace("'", "'"))

fig, ax = plt.subplots(1,2,figsize=(10,5))
epochs = range(len(history_dict["loss"]))

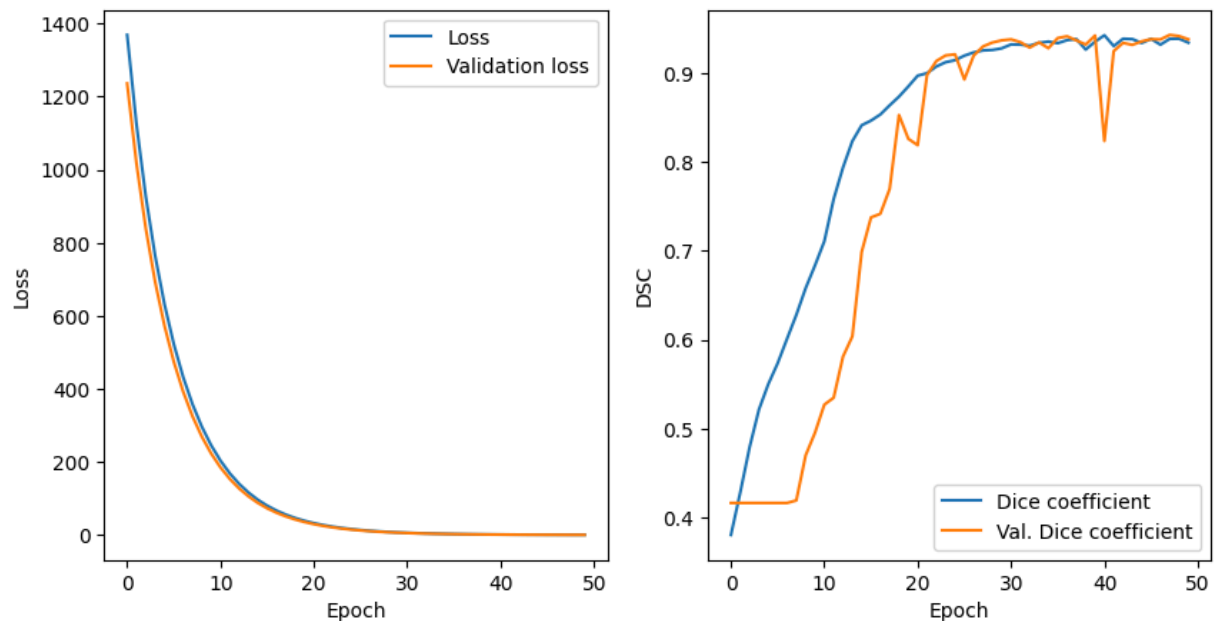
ax[0].plot(epochs, history_dict["loss"], label="Loss")
ax[0].plot(epochs, history_dict["val_loss"], label="Validation loss")
ax[1].plot(epochs, history_dict["dice_coef"], label="Dice coefficient")
ax[1].plot(epochs, history_dict["val_dice_coef"], label="Val. Dice coefficient")

ax[0].set_xlabel("Epoch")
ax[0].set_ylabel("Loss")
ax[1].set_xlabel("Epoch")
ax[1].set_ylabel("DSC")

ax[0].legend()
ax[1].legend()

#ax[0].set_yscale("Log")

plt.show()
plt.close()
```



As you can see from this images, the training seems to have gone smoothly. Just a reminder:

- *How overfitting would look:* The blue line (training) would continue to drop while the orange line (validation) hits a minimum and then starts to rise again, indicating the model is memorizing the training data but failing on new data.
- *How underfitting would look:* Both lines would remain high and flat, meaning the model isn't learning much at all.

Now that our training is complete, it is time to perform some predictions and store the results.

NB: the loaded numpy array has the following dimension (x, y, c) , where x and y are the size of the image while c is the number of channels. The keras `predict` method reads multiple images at once and in this example we'll iterate through the images and infer the results one image at a time. So in this case we should pass to `predict` an image with shape $(1, x, y, c)$.

For completeness, we will also be using another metric called **Boundary Dice Similarity Coefficient**. While the standard Dice score evaluates the overall overlap of the masks, the Boundary Dice specifically measures how accurately the edges (contours) of your predicted mask match the edges of the ground truth mask.

```
In [27]: # Cell to define the Border Dice written by Gemini PRO

from scipy.ndimage import binary_erosion

def compute_boundary(mask):
    """Extracts the 1-pixel thick boundary of a binary mask."""
    # Ensure the mask is boolean (True/False)
    mask = mask.astype(bool)
    # Erode the mask by 1 pixel
    eroded = binary_erosion(mask)
    # The boundary is the original mask MINUS the eroded mask (XOR operation)
    return mask ^ eroded

def border_dice(y_true, y_pred, smooth=1e-6):
    """Computes the strict Boundary Dice Similarity Coefficient."""
    # Extract boundaries
    b_true = compute_boundary(y_true)
    b_pred = compute_boundary(y_pred)

    # Calculate the Dice formula strictly on the boundary pixels
    intersection = np.sum(b_true & b_pred)
    cardinal_true = np.sum(b_true)
    cardinal_pred = np.sum(b_pred)

    return (2. * intersection + smooth) / (cardinal_true + cardinal_pred + smooth)
```

```
In [28]: test_list = data_list[590:]

dice = []
bdsc = []
pred = []
```

```

for patient in test_list:
    #print(patient)
    data = np.load(path + patient)
    img = np.expand_dims(data[:, :, 0], axis=-1)
    img = np.expand_dims(img, axis=0)

    #print(img.shape)
    label = data[:, :, 1]
    label = np.expand_dims(data[:, :, 1], axis=-1)
    label = np.expand_dims(label, axis=0)

    # Actually predict
    prediction = unet.predict(img, verbose=0)
    prediction = np.round(prediction)
    #print(prediction.shape)
    pred.append(prediction)
    dice.append(dice_coef(label.astype('float32'), prediction.astype('float32')))
    bdsc.append(border_dice(label[0, :, :, 0], prediction[0, :, :, 0]))

    #img_label = Image.fromarray((prediction[0, :, :, 0]*255)).convert('L')
    #img = Image.fromarray((img[0, :, :, 0]*255)).convert('L')

    #img.paste(img_label, (0,0), mask = img_label)
    #img_name = '../prediction_png/' + patient[:-4] + '.png'
    #img.save(img_name)

avg_dice = np.mean(dice)
std_dice = np.std(dice)
avg_sdsc = np.mean(bdsc)
std_sdsc = np.std(bdsc)

print('*****+')
print('Mean DSC', avg_dice, '+-', std_dice)
print('Mean BDSC', avg_sdsc, '+-', std_sdsc)
print('*****+')

```

```

*****+
Mean DSC 0.9459807 +- 0.020855296
Mean BDSC 0.12235035928863251 +- 0.04572889263365931
*****+

```

Now we'll plot the best and worst case for different metrics (i.e. Dice and border Dice). To do this we define two function which will find the index of best and worst case and a third function which plots the predicted and real mask next to each other to appreciate the differences. We will also plot a random predicted mask just for completeness.

```

In [29]: def get_minvalue(inputlist):

    #get the minimum value in the list
    min_value = min(inputlist)

    #return the index of minimum value
    min_index=inputlist.index(min_value)
    return min_index

def get_maxvalue(inputlist):

    #get the minimum value in the list
    min_value = max(inputlist)

    #return the index of minimum value
    min_index=inputlist.index(min_value)
    return min_index

def plot_compare(index, metric=""):
    fig, ax = plt.subplots(1,2,figsize=(10,5))

    fig.suptitle(metric)

    data = np.load(path + test_list[index])
    mask_true = np.ma.masked_where(data[:, :, 1]<1,data[:, :, 1])
    mask_pred = np.ma.masked_where(pred[index][0, :, :, 0]<1,pred[index][0, :, :, 0])

    ax[0].imshow(data[:, :, 0], cmap="bone")
    ax[0].imshow(mask_true, cmap="Reds", vmin=0, vmax=1)
    ax[0].axis('off')
    ax[0].set_title('Real mask')

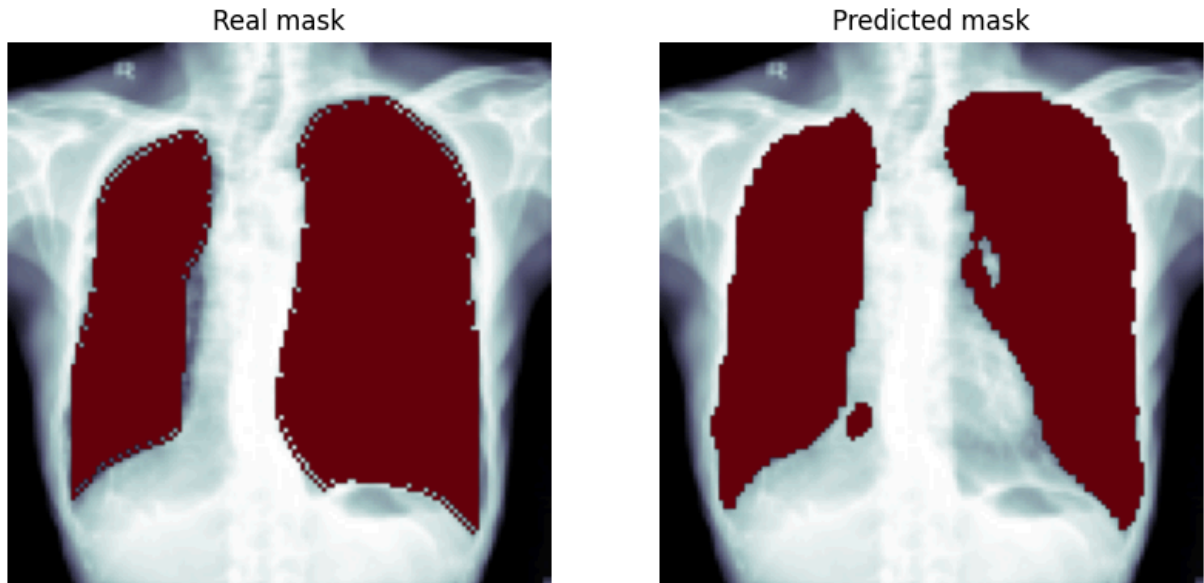
```

```
ax[1].imshow(data[...,:0], cmap="bone")
ax[1].imshow(mask_pred, cmap="Reds", vmin=0, vmax=1)
ax[1].axis('off')
ax[1].set_title('Predicted mask')

plt.show()
plt.close()
```

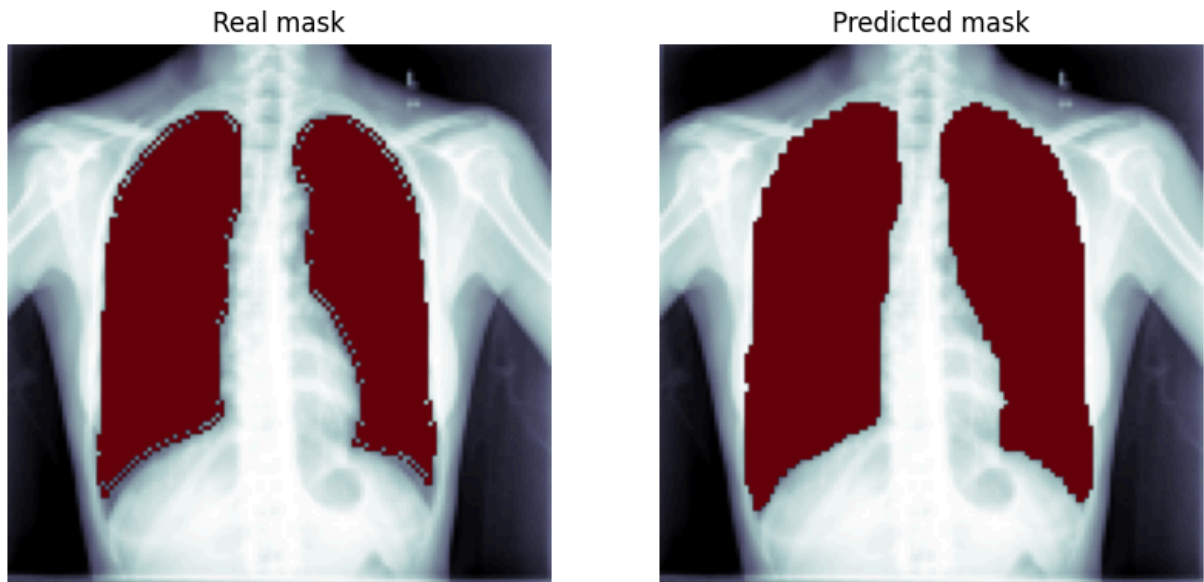
```
In [30]: min_dsc = get_minvalue(dice)
plot_compare(min_dsc, metric="Dice = {}".format(dice[min_dsc]))
```

Dice = [0.85378814]



```
In [31]: max_dsc = get_maxvalue(dice)
plot_compare(max_dsc, metric="Dice = {}".format(dice[max_dsc]))
```

Dice = [0.9704336]



```
In [32]: fig, ax = plt.subplots(1,2,figsize=(10,5))

data = np.load(path + test_list[4])
fig.suptitle("Dice = {}".format(dice[4]))
mask_true = np.ma.masked_where(data[...,:1]<1,data[...,:1])
mask_pred = np.ma.masked_where(pred[4][0,...,:0]<1,pred[4][0,...,:0])

ax[0].imshow(data[...,:0], cmap="bone")
ax[0].imshow(mask_true, cmap="Reds", vmin=0, vmax=1)
ax[1].imshow(data[...,:0], cmap="bone")
ax[1].imshow(mask_pred, cmap="Reds", vmin=0, vmax=1)
```

```
plt.show()  
plt.close()
```

Dice = [0.9634454]

